

GradeOps

AI-Powered Human-in-the-Loop Exam Grading Pipeline

Technical Report

Even Semester Projects 2026

IIT Guwahati Coding Club

Team Members: Aditya Om Sah
Mallhar Totey

Repository: <https://github.com/adityaomsah/GradeOps>

Live Backend: <https://gradeops-backend-06ww.onrender.com>

Live Frontend: <https://grade-jguwjxw0z-adityaomsah1.vercel.app>

Date: June 2026

Abstract

GradeOps is a Human-in-the-Loop (HITL) AI grading pipeline that automates the evaluation of handwritten exam scripts using Vision-Language Models (VLMs) and Agentic Large Language Models. The system allows instructors to upload bulk exam scans and JSON rubrics, automatically extract and transcribe handwritten answers, award partial credit with structured justifications, flag plagiarism, and push AI-proposed grades to a TA review dashboard for final approval or override. This report documents the problem statement, technical architecture, implementation details, challenges encountered, and future roadmap.

Contents

1	Problem Statement and Context	2
1.1	The Problem	2
1.2	Project Specification	2
1.3	Team Division	2
2	What We Built	3
2.1	System Overview	3
2.2	Features Implemented	3
2.2.1	Authentication and Role-Based Access Control	3
2.2.2	OCR Pipeline	3
2.2.3	LangGraph Grading Pipeline	4
2.2.4	Plagiarism Detection	4
2.2.5	API Endpoints	4
2.2.6	Database Schema	5
2.2.7	Audit Trail	5
2.2.8	Frontend	5
3	Technical Architecture	6
3.1	Backend Stack	6
3.2	Frontend Stack	6
3.3	Deployment Architecture	6
4	Errors Encountered and How We Handled Them	7
4.1	Error 1: bcrypt / passlib Version Incompatibility	7
4.2	Error 2: datetime.utcnow() Deprecation	7
4.3	Error 3: @app.on_event("startup") Deprecation	7
4.4	Error 4: Neon PostgreSQL + psycopg2 Unknown PG numeric type: 3802	8
4.5	Error 5: Swagger UI OAuth2 Form vs JSON Body Mismatch	8
4.6	Error 6: CORS Blocking Frontend Requests	9
4.7	Error 7: Qwen2-VL Model Loading on CPU	9
5	Future Goals	9
5.1	Short-Term (Next Month)	9
5.2	Medium-Term (Next Quarter)	10
5.3	Long-Term (Production Deployment)	10
6	Conclusion	10
A	API Quick Reference	10
B	Environment Variables	11

Problem Statement and Context

The Problem

Grading handwritten exams in university settings is a time-consuming, labour-intensive, and inconsistency-prone process. At institutions like IIT Guwahati with large class sizes, evaluating hundreds of answer scripts manually results in:

- **Inconsistency:** Different TAs applying the same rubric differently across scripts.
- **Fatigue-induced bias:** Graders become less rigorous after evaluating many scripts in succession.
- **Slow turnaround:** Results taking days or weeks to be published.
- **Lack of traceability:** No structured audit trail for score changes or TA overrides.
- **Plagiarism blind spots:** Manual detection of academic dishonesty is unreliable at scale.

Project Specification

The Even Semester Projects 2026 brief from IIT Guwahati Coding Club specified the following goals for GradeOps:

Official Goals

1. A web portal to upload bulk exam scans (PDFs) and define granular grading rubrics.
2. Role-Based Access Control (Instructors vs. TAs).
3. Specialized OCR/Vision models (Nougat/Qwen-VL) to extract handwritten answers and store them in cloud storage.
4. An Agentic LLM pipeline to:
 - Award partial credit based on strict rubric conditions.
 - Generate structured textual justifications for each score.
 - Flag highly similar answers across papers for plagiarism.
5. A high-throughput review dashboard with keyboard shortcuts for rapid TA approvals and overrides.

Prescribed tech stack: Python, PyTorch, Hugging Face (Nougat/Qwen-VL), LangChain, LangGraph, React.js, FastAPI, PostgreSQL.

Team Division

The project was divided as follows:

Component	Owner	Notes
Backend API (FastAPI)	Aditya Sah	All 16 endpoints
ML/Grading Pipeline	Aditya Sah	LangGraph + Gemini 2.0 Flash
Plagiarism Detection	Aditya Sah	Jaccard + Cosine hybrid
Database Models	Mallhar Totey	SQLAlchemy ORM
React Frontend	Mallhar Totey	Full dashboard
Deployment	Both	Render + Vercel

What We Built

System Overview

GradeOps implements a complete Human-in-the-Loop grading pipeline. Figure ?? illustrates the high-level data flow.

High-Level Pipeline

PDF Upload → OCR Extraction → Plagiarism Check → LLM Grading → TA Review Dashboard → Results

Features Implemented

Authentication and Role-Based Access Control

A four-tier RBAC system was implemented using JWT (JSON Web Tokens) via `python-jose` and `passlib` with `bcrypt` password hashing.

Role	Permissions
Admin	Register all user types; system-level management
Instructor	Create exams and rubrics; view analytics, results, and history; approve/override grades
TA	Upload exam PDFs; trigger grading (single and bulk); view and export results; approve/override grades
Student	View own graded result only

Tokens expire after 24 hours. The first admin is auto-created from environment variables on server startup using the FastAPI `lifespan` event (replacing the deprecated `@app.on_event` pattern).

OCR Pipeline

Answer scripts are uploaded as PDFs and converted to page images using `pdf2image` (backed by `poppler`). Each page image is passed to a vision extractor that returns structured JSON:

```
1 {  
2   "name": "John Doe",  
3   "roll_no": "101",  
4   "raw_text": "Dynamic memory allocation uses malloc()..."  
5 }
```

Listing 1: Vision Extractor return format

Dual-mode implementation:

- **Local (GPU):** Qwen2-VL-2B-Instruct via Hugging Face Transformers — implements the exact VLM pipeline prescribed in the project spec, with structured JSON prompting, markdown fence stripping, and graceful JSON fallback.
- **Deployment (CPU):** Gemini Vision API via `langchain-google-genai` — functionally equivalent, no GPU required.

LangGraph Grading Pipeline

The grading pipeline is implemented as a directed acyclic graph using LangGraph with conditional routing:

```

1 class GradeState(TypedDict):
2     student_name: str
3     student_roll_no: str
4     answer_script: str
5     rubric: str          # JSON-encoded rubric
6     score: float
7     justification: str
8     all_answers: List[str] # for plagiarism
9     plagiarism_score: float
10    plagiarism_flag: bool

```

Listing 2: GradeState TypedDict structure

Pipeline flow:

1. **plagiarism_check**: Computes Jaccard similarity and cosine similarity (via `sentence-transformer` model `all-MiniLM-L6-v2`) against all other submitted answers. Answers shorter than 50 words skip the check. Flag is raised if `jaccard > 0.6 AND cosine > 0.85`.
2. **Conditional routing**: Flagged answers route to `human_review_needed` (score set to `-1`, justification contains warning). Clean answers proceed to grading.
3. **grade_answer**: Gemini 2.0 Flash evaluates the answer against rubric criteria and returns a structured `GradingResponse` Pydantic object.
4. **generate_justification**: Gemini generates a free-text explanation of the awarded marks.

Plagiarism Detection

The hybrid Jaccard + Cosine approach was chosen after the team identified that cosine similarity alone produces false positives (semantically similar but independently written answers). The dual-threshold system significantly reduces false flags:

$$\text{flag} = \begin{cases} \text{True} & \text{if } J(A_i, A_j) > 0.6 \wedge \cos(A_i, A_j) > 0.85 \\ \text{False} & \text{otherwise} \end{cases} \quad (1)$$

where J is Jaccard similarity over word sets and $\cos$ is cosine similarity over sentence embeddings.

API Endpoints

The backend exposes 19 REST API endpoints:

Endpoint	Method	Roles	Purpose
/login	POST	All	JWT token issuance
/register	POST	Admin	User account creation
/upload-pdf/	POST	TA	PDF upload + image conversion
/rubric	POST	Instructor	Rubric PDF upload + LLM parsing
/rubric/{id}	GET	Instructor, TA	Rubric retrieval
/exam	POST	Instructor	Exam creation
/exam	GET	Instructor, TA	List all exams
/exam/{id}	GET	Instructor, TA	Single exam retrieval
/grade	POST	TA	Single PDF grading
/grade/bulk	POST	TA	Multi-PDF batch grading
/results	GET	Instructor, TA	All graded results
/results/{roll_no}	GET	All	Student result (own only for student)
/results/{id}/history	GET	Instructor, TA	Grade audit trail
/results/export/csv	GET	Instructor, TA	CSV download
/feedback	POST	Instructor, TA	Approve or override grade
/submissions	GET/POST	Instructor, TA	Submission tracking
/submissions/{id}	GET/PATCH	Instructor, TA	Single submission
/analytics/dashboard	GET	Instructor, TA	Global stats summary
/analytics/exam/{id}	GET	Instructor, TA	Per-exam analytics

Database Schema

Six SQLAlchemy models back the system, targeting PostgreSQL (Neon) in production and SQLite locally:

- **User:** id, email, password_hash, role, roll_no
- **Exam:** id, course_name, course_code, exam_type, total_marks
- **Rubric:** id, question, max_marks, criteria (JSON)
- **Grade:** id, exam_id, rubric_id, student_name, student_roll_no, score, justification, plagiarism_score, plagiarism_flag, ta_reviewed, ta_override_score, uploaded_pdf_file
- **GradeHistory:** id, grade_id, old_score, new_score, changed_by, reason, action, timestamp
- **Submission:** id, exam_id, student_roll_no, file_url, status, grade_id, score_cache, created_at

Audit Trail

Every grade change is logged to **GradeHistory** with the action type (**ai_graded**, **approved**, **overridden**), before/after scores, reviewer identity, timestamp, and justification. This enables full traceability for academic integrity purposes.

Frontend

The React frontend (Vite + Tailwind CSS) provides:

- **Dark/Light/System theme** switcher with localStorage persistence.
- **Role-aware navigation:** Different pages visible per role.
- **Dashboard** with live analytics cards (total exams, graded submissions, plagiarism flags, pending reviews).
- **Exam management:** Create and list exams (Instructor).
- **Rubric portal:** Upload PDFs, view parsed criteria, session-cached recent rubrics.

- **Upload & Grade:** Single PDF and bulk multi-PDF upload with per-file result display.
- **Results dashboard:** Filter by pending/flagged/reviewed, inline approve/override with justification, audit history panel.
- **Analytics:** Global dashboard summary, per-exam score stats, TA review progress bars.
- **Student result page:** Score display with status indicator.
- **Admin register page:** Create all user types with roll number support for students.

Technical Architecture

Backend Stack

Layer	Technology
Web Framework	FastAPI (Python 3.11+)
Agent Orchestration	LangGraph 0.1+
LLM	Gemini 2.0 Flash (<code>langchain-google-genai</code>)
Embeddings	<code>sentence-transformers</code> (all-MiniLM-L6-v2)
OCR (Local)	Qwen2-VL-2B-Instruct (Hugging Face)
OCR (Deploy)	Gemini Vision API
Database ORM	SQLAlchemy 2.0
Database (Local)	SQLite
Database (Production)	Neon PostgreSQL (serverless)
Authentication	JWT (<code>python-jose</code>) + <code>passlib/bcrypt</code>
PDF Processing	<code>pdf2image</code> + <code>poppler</code>
Data Validation	Pydantic v2

Frontend Stack

Layer	Technology
Framework	React 18 (Vite)
Styling	Tailwind CSS 3
Routing	React Router v6
HTTP Client	Axios (with interceptors)
State Management	React Context API
Theme	<code>class</code> dark mode strategy

Deployment Architecture

Service	Platform	URL
Frontend (React)	Vercel	<code>grade-jguwjxw0z-adityaomsah1.vercel.app</code>
Backend (FastAPI)	Render	<code>gradeops-backend-06ww.onrender.com</code>
Database	Neon (PostgreSQL)	Serverless pool

Errors Encountered and How We Handled Them

Error 1: bcrypt / passlib Version Incompatibility

Error

ValueError: password cannot be longer than 72 bytes and
AttributeError: module 'bcrypt' has no attribute '__about__'

Root cause: passlib 1.7.x was built against older bcrypt versions and reads `bcrypt.__about__.__version__` for version detection. Newer bcrypt versions removed this attribute, causing passlib to fall into an error state that also triggered the 72-byte limit incorrectly.

Fix

Pin `bcrypt==4.0.1` in `requirements.txt`. This is the last version fully compatible with passlib 1.7.4 without requiring patching. The constraint is documented with a comment in requirements.

Error 2: `datetime.utcnow()` Deprecation

Error

DeprecationWarning: `datetime.utcnow()` is deprecated in JWT token generation and GradeHistory timestamps.

Root cause: Python 3.12+ deprecated `datetime.utcnow()` in favour of timezone-aware datetimes.

Fix

Replaced all occurrences with `datetime.now(timezone.utc)` and imported `timezone` from `datetime`. For SQLAlchemy columns, used `server_default=func.now()` with `DateTime(timezone=True)` to delegate timestamp generation to the database.

Error 3: `@app.on_event("startup")` Deprecation

Error

FastAPI emitted deprecation warnings for `@app.on_event("startup")` in newer versions.

Root cause: FastAPI moved to the `lifespan` context manager pattern as the canonical way to handle startup/shutdown events.

Fix

Migrated to `@asynccontextmanager` lifespan pattern:

```
1 from contextlib import asynccontextmanager
2
```



```
3 @asynccontextmanager
4 async def lifespan(app: FastAPI):
5     # startup: create tables, seed admin
6     models.BaseClass.metadata.create_all(bind=engine)
7     _seed_admin(next(get_db()))
8     yield # app runs here
9     # shutdown cleanup (if needed)
10
11 app = FastAPI(title="GradeOps API", lifespan=lifespan)
```

Error 4: Neon PostgreSQL + psycopg2 Unknown PG numeric type: 3802

Error

sqlalchemy.exc.InvalidRequestError: Unknown PG numeric type: 3802 when querying the submissions table through Neon's connection pooler.

Root cause: OID 3802 is jsonb. Neon uses PgBouncer in transaction-mode pooling, which can return stale prepared statement plans from a previous connection's type cache that reference jsonb (used in the Rubric.criteria column). When a new connection handles a query for a different table, psycopg2's local type registry doesn't recognise the OID from the cached plan.

Fix

Added pool_pre_ping=True and pool_recycle=300 to the SQLAlchemy engine. For a more robust long-term fix, the DATABASE_URL for Neon should append ?options=-c%20plan_cache_mode=force_custom_statement to disable prepared statement caching. Alternatively, switching to psycopg v3 (instead of psycopg2-binary) handles Neon's pooler correctly out of the box.

Error 5: Swagger UI OAuth2 Form vs JSON Body Mismatch

Error

422 Unprocessable Entity when using the Swagger UI Authorize button with the /login endpoint.

Root cause: OAuth2PasswordBearer in FastAPI's Swagger UI sends login credentials as application/x-www-form-urlencoded with a username field, but the original /login implementation used a Pydantic JSON body with an email field.

Fix

Changed /login to use OAuth2PasswordRequestForm = Depends() which reads username from form data. The username field still receives email values — the naming is a FastAPI/OAuth2 convention. This made the Swagger UI Authorize button work correctly and aligned with OAuth2 standards.

Error 6: CORS Blocking Frontend Requests

Error

Access to XMLHttpRequest at 'http://127.0.0.1:8000/...' from origin 'http://localhost:5173' has been blocked by CORS policy.

Root cause: The FastAPI backend did not include `CORSMiddleware`, so all cross-origin requests from the React dev server were rejected by the browser.

Fix

Added `CORSMiddleware` to `main.py` with explicit allowed origins for both local development (`localhost:5173`) and the production Vercel domain.

Error 7: Qwen2-VL Model Loading on CPU

Error

Server tried to download and load the 4.42 GB Qwen2-VL model on startup (with `device_map="auto"` mapping to CPU), causing extremely slow startup and OOM errors on deployment machines.

Root cause: The `vision_extractor.py` import was at the module level, triggering model download on every server start regardless of environment.

Fix

For development: comment out all model loading code, use a mock function returning static test data. For production deployment: implement `vision_extractor_gemini.py` using the Gemini Vision API (already available via `GEMINI_API_KEY`), which requires no local GPU. The original Qwen2-VL implementation is preserved and documented for GPU-equipped environments.

Future Goals

Short-Term (Next Month)

1. **Cloudinary/S3 Integration:** Store uploaded PDF images in cloud storage and expose URLs in the Grade record, enabling the TA dashboard to display the original answer scan side-by-side with the AI grade — a core feature from the project spec not yet fully implemented.
2. **Keyboard Shortcuts in TA Dashboard:** Add `A` to approve and `O` to open override dialog for rapid review — as specified in the project brief.
3. **Grade Distribution Analytics:** Implement `GET /analytics/grade-distribution` returning A/B/C/D/F bucketed counts across all exams, with frontend visualisation.
4. **psycopg v3 Migration:** Migrate from `psycopg2-binary` to `psycopg` (v3) for better Neon compatibility and native async support.

Medium-Term (Next Quarter)

1. **Async Grading Queue:** Introduce Celery + Redis for background grading jobs, returning a `job_id` immediately and pushing status updates via WebSocket or polling. This eliminates HTTP timeout issues for large batches.
2. **Rubric Versioning:** Support multiple rubric versions per question, allowing instructors to refine criteria mid-session without losing historical grading context.
3. **Bulk PDF Splitting:** Automatically segment a combined PDF (all students in one file) into per-student scripts using roll number detection from the OCR output.
4. **Email Notifications:** Notify students when their results are published and TAs when new submissions arrive for review.
5. **Mobile-Responsive TA Dashboard:** Optimise the review interface for tablet use so TAs can grade on the go.

Long-Term (Production Deployment)

1. **GPU Inference Service:** Deploy Qwen2-VL on a GPU-enabled instance (AWS EC2 g4dn.xlarge or RunPod) and expose it as a microservice, replacing the Gemini Vision fallback for cost efficiency at scale.
2. **Multi-Subject Rubric Library:** Allow instructors to build and reuse rubric templates across semesters.
3. **LMS Integration:** Connect with Moodle or the institute's existing Learning Management System via webhooks for automatic grade sync.
4. **Dispute Resolution Portal:** Allow students to formally contest AI or TA grades through a structured re-evaluation workflow.
5. **Differential Privacy:** Aggregate analytics computed with differential privacy guarantees to protect individual student data.

Conclusion

GradeOps successfully demonstrates the feasibility of AI-assisted exam grading with robust human oversight. Within the Even Semester 2026 project window, the team delivered a full-stack production-deployed system encompassing a 19-endpoint FastAPI backend, a LangGraph-powered agentic grading pipeline, hybrid plagiarism detection, a four-role RBAC system, a complete React dashboard with dark mode, and live deployment on Render and Vercel.

The project exceeded the original specification's minimum requirements in several areas — notably the addition of a full audit trail (**GradeHistory**), CSV export, bulk grading, submission tracking, and analytics endpoints not originally prescribed. The architectural decisions made throughout (hybrid plagiarism detection, optional submission tracking layer, dual OCR modes) reflect real-world engineering trade-offs between correctness, deployment practicality, and development velocity.

The codebase is open source at <https://github.com/adityaomsah/GradeOps>.

API Quick Reference

All endpoints are accessible via the deployed backend at <https://gradeops-backend-06ww.onrender.com/docs> (Swagger UI).

Authentication: All endpoints except POST /login require
Authorization: Bearer <token> header.

Environment Variables

```
1 # Database
2 DATABASE_URL=postgresql://user:pass@host/dbname
3
4 # JWT
5 SECRET_KEY=<random 64-char hex string>
6
7 # LLM
8 GEMINI_API_KEY=<your Gemini API key>
9
10 # First admin bootstrap
11 ADMIN_EMAIL=admin@gradeops.com
12 ADMIN_PASSWORD=<secure password>
```

Listing 3: Required .env variables